Adaptive Chunk-Based Compression for Inverted Index Posting Lists

CSCE 410/810 — Information Retrieval

University of Nebraska–Lincoln

Cooper Malone

April 21, 2026

Table of Contents

Table of Contents 2
1. Background 3
2. Innovation 3
3. Significance 3
4. Feasibility 4
5. Evaluation 4
6. Data 5
6.1 Dataset Description 5
6.2 Preprocessing 5
7. Methodology 5
7.1 vByte Compression 5
7.2 Elias Gamma Compression 6
7.3 BitPacking Compression 6
7.4 Fixed-Chunk Compression 6
7.5 Adaptive-Chunk Compression 6
8. Results 7
8.1 Natural Compression Ratio 7
8.2 Synthetic Compression Ratio 8
8.3 Compression Time 9
9. Conclusion & Future Work 10

1. Background

Integer compression is widely used in systems such as databases and search engines to reduce storage and improve data transfer efficiency. One of the simplest and most common techniques is Variable Byte (vByte) encoding, which represents integers using a variable number of bytes. Each byte contributes seven bits to the value, with an additional control bit indicating whether more bytes follow. This approach is efficient for small values and offers fast decoding, but it operates on individual integers and does not take advantage of patterns across sequences of data.

To improve compression performance, chunk-based algorithms group integers into blocks and encode them collectively. This allows the compression scheme to exploit similarities within each chunk, often achieving better compression ratios and improved cache efficiency. Adaptive chunk-based methods extend this idea by dynamically selecting chunk sizes or encoding strategies based on the data distribution. The algorithm developed in this project builds on both vByte encoding and chunk-based compression by combining variable-length encoding with adaptive chunking to balance compression efficiency and decoding speed.

2. Innovation

The innovation in this project is in its approach towards finding the best way to compress inverted indexes. The method we propose will be an automated way to determine the most efficient way to compress indexes and provide comparisons between several methods. This differs from other approaches as we analyze the data with multiple methods first, before deciding on a final method that will best compress on index before finalizing the index compression.

3. Significance

Modern information retrieval systems rely heavily on inverted indexes, this frequently generates massive posting lists. These lists are compressed using gap encoding algorithms like standard vByte, which compresses the data integer by integer. This applies a one size fits all approach by assuming that a consistent distribution of gaps exists throughout the dataset. In actual applications, posting lists are highly diverse, containing both dense regions characterized by small gaps and sparse regions characterized by large gaps. Applying a rigid compression strategy makes static methods fail to account for varying data distributions, ultimately wasting storage space with suboptimal decoding speeds when processing sparse, scattered data.

To address the inefficiencies of static encoding, this project explores and evaluates the implementation of adaptive chunk-based compression. The primary impact of this project lies in the optimization of the trade-off that fundamentally exists in information retrieval. By developing a context aware system that dynamically adjusts block sizes and compression algorithms based on the specific data, we can minimize the storage footprint of the index while maximizing query decoding speed. Dynamically matching the appropriate compression technique to the context of the data eliminates the need to compromise between storage efficiency and system production. This research demonstrates that flexible, context aware compression models can achieve both a highly compressed index footprint and rapid search engine performance. Overall, this provides a scalable solution for managing increasingly large real world datasets.

4. Feasibility

This project was extremely feasible. We had four main factors that made us believe that we could execute this project in the time we were provided. The factors were: past experience, innovation, adaptability, and desired outcome.

First of all, we looked at our past experiences with vByte chunk compression. During our sixth assignment, we got hands-on experience with vByte and quickly learned how it worked. We were able to test compression on data with different sizes of gaps between them. This would later help bring us to the idea we had for this project: adaptive chunk based compression. This experience directly led to us being able to work on this project in the first place.

Second, we looked at the innovation that we could create. We compared vByte to normal chunk-based compression and saw that we could create something that is more flexible than both of them, even if it would cost more computer resources. In terms of CPU overhead, vByte is known for having a low overhead while normal chunk based has a medium overhead. Both of those are quite rigid as opposed to the flexible algorithm we wanted to create.

Third, we realized that adapting based on what we knew of vByte and chunk-based. Both methods have similar principles in how they handle compression, but they each have their own differences. These differences in design let us create an adaptive algorithm that would build upon the principles held within both, ensuring that we could optimize compression time and ratio.

Finally, the outcome we sought after was rather feasible as we could easily see the gap in normal vByte and chunk-based compression. Due to this, we believed that our idea of combining them into an adaptive algorithm would be rather simple to create. After all, all we needed to do was built upon what already existed through utilizing information we had already learned.

5. Evaluation

We evaluate our proposed adaptive chunk-based compression algorithm against two baselines: vByte encoding and a standard fixed chunk-based compression method. All algorithms were implemented within the same codebase and executed on identical hardware to ensure a fair comparison. Experiments were conducted in a controlled environment with no competing processes, and each result was averaged over multiple runs to reduce noise and account for runtime variability.

To ensure robustness, we tested on a diverse collection of datasets representing different data characteristics. Dataset sizes were also varied to evaluate performance at different scales. This diversity is important because compression performance for adaptive methods can vary significantly depending on the distribution of locality of patterns within the data.

We measure performance using several key metrics. Compression ratio is used to quantify space efficiency, defined as the size of the compressed output relative to the original input. Compression time and decompression time capture the computational cost of each method. To better understand trade-offs between space-saving and speed, we also report an overall efficiency metric, defined as compression ratio normalized by total processing time. Beyond these primary metrics, we analyze scalability by measuring how performance changes as the input size increases. We also evaluate the sensitivity of the adaptive algorithm to its internal parameters, such as chunk size boundaries and adaptation frequency, to understand how these choices impact performance.

6. Data

6.1 Dataset Description

We use the following datasets for experiments:

Dataset	Documents	Source
The Complete Works of William Shakespeare by William Shakespeare	194	https://www.gutenberg.org/ebooks/100
6 Newsgroups	1928	http://qwone.com/~jason/20Newsgroups/
Simulated Gap Data	0	Generated Randomly

6.2 Preprocessing

For the document based datasets such as “The Complete Works of William Shakespeare” and the “6 Newsgroups” the data was preprocessed before being used for comparison. This preprocessing included a pipeline to turn documents into a posting list. First, the documents are broken down into tokens by splitting to text on every space, this creates tokens from all words in the document. Then, tokens undergo the normalization step of converting all tokens in the documents into lowercase. Next, the terms and document IDs are added to the posting lists, ensuring that there are no duplicates. Then, if there are multiple documents, all documents are aggregated into a single posting list, repeating all the previous steps. Finally, the gaps between consecutive positions are calculated to help facilitate our calculations.

The generated datasets used for gap sized based compression do not require preprocessing as they only simulate gap sizes and are not created from any real documents.

7. Methodology

7.1 vByte Compression

This implemented methodology applies variable-byte (vByte) encoding to compress integer sequences, particularly optimized for posting lists in information retrieval systems. The encoder operates by decomposing each non-negative integer into 7-bit segments using bitwise operations. For each segment, the most significant bit is used as a continuation flag. It is set to 1 for all preceding bytes and 0 for the final byte, indicating termination. This process is implemented iteratively by masking the lower 7 bits and right-shifting the value until all segments are processed. Encoded integers are concatenated to form a compact byte stream, enabling efficient storage of variable-length representations.

7.2 Elias Gamma Compression

The next implementation involves a custom Elias gamma encoding scheme to compress integer sequences at the bit level, enabling more compact representations than byte-aligned methods. Each non-negative integer is first transformed by adding one to ensure compatibility with the encoding format. The encoded representation consists of two components: a unary prefix and a binary suffix. The unary prefix encodes the length of the binary representation using a sequence of zeros followed by a one, while the suffix stores the remaining bits of the value with its leading one. Our implementation explicitly manages bit-level packing through a buffered writing procedure, allowing encoded values to span across byte boundaries and minimizing wasted space. Sequences of integers are encoded consecutively into a continuous bitstream, with the remaining bits padded to complete the final byte.

7.3 BitPacking Compression

The BitPacking implementation involves a custom bit-packing compression scheme that encodes integer sequences using a fixed number of bits determined by the data distribution. For each sequence, the minimum value is first identified and subtracted from all elements to produce a set of non-negative deltas. The maximum delta is then used to compute the required bit width ensuring that each value is represented using the smallest uniform number of bits. The encoded output begins with a compact header containing the minimum value, bit width, and sequence length, followed by a densely packed bitstream where values are written consecutively into a buffer and flushed into bytes as capacity is reached. This design minimizes storage overhead by eliminating unused bits typically present in fixed-width representations.

7.4 Fixed-Chunk Compression

The fixed-chunk compression framework segments integer sequences into uniform, contiguous blocks prior to applying compression. Given an input sequence, a predefined chunk size is used to partition the data into equally sized sublists, with the final chunk potentially smaller if the sequence length is not divisible by the chunk size. Each chunk is then independently compressed using a configurable compression function, enabling modular integration with different encoding schemes. This design allows compression to operate on smaller, localized subsets of data, which can improve cache efficiency and enable selective decompression of only the required portions.

7.5 Adaptive-Chunk Compression

Finally, the adaptive chunk-based compression algorithm works by first dividing a sequence of integers into fixed-size chunks, then dynamically selecting the most space-efficient encoding method for each chunk independently. For every chunk, it evaluates multiple candidate codecs by compressing the data with each and estimating the resulting size using a unified metric. The codec that produces the smallest representation is chosen and recorded alongside the compressed data, enabling heterogeneous encoding across chunks. This localized decision-making allows the algorithm to adapt to variations in data distribution, such as dense vs sparse regions within the same list. During decompression, the process is reversed by reading the stored codec identifier for each chunk and applying the corresponding decoding function, after which all chunks are concatenated to reconstruct the original sequence.

8. Results

8.1 Natural Compression Ratio

The results were first created by evaluating the compression ratio of the various algorithms on a well known piece of text. Fixed chunk compression was omitted from these results, as it never performed better than the best algorithms and was instead a critical component of making the adaptive chunk-based algorithm. The first test involved testing the compression ratio on a well known piece of text. In Figure 1, the compression ratio between vByte, Elias, Bitpack, and Adaptive chunking algorithms can be seen. vByte and Adaptive chunking consistently perform the best, with an average compression ratio between 40% to 50%. While being extremely close, the Adaptive chunking algorithm always performs slightly better than vByte no matter the chunk size. The Elias and BitPack methods are much less efficient on a generic dataset such as Shakespeare. However, instances in the data allow Adaptive Chunking algorithms to choose these more inefficient algorithms when necessary.

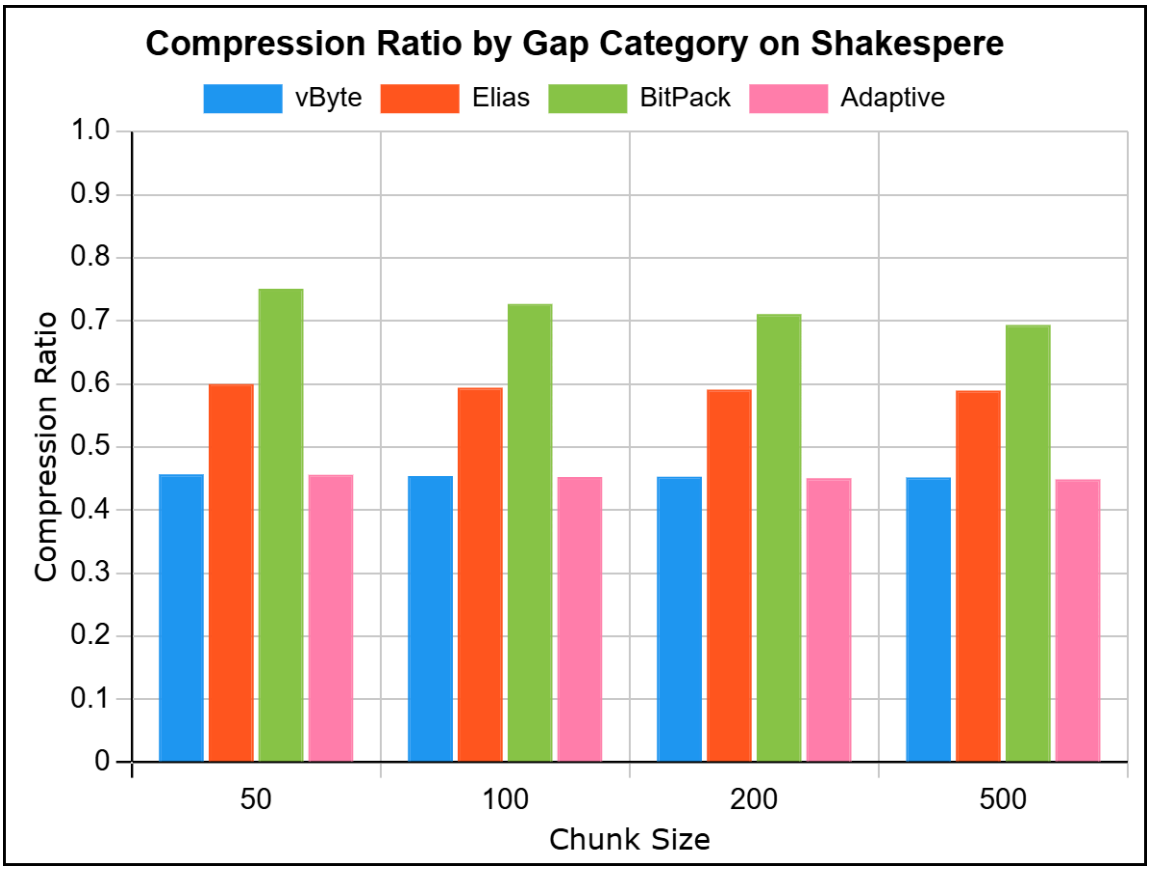


Figure 1. Compression ratio size by chuck size for each compression method

Next, a more sparse real world dataset called Newsgroups was chosen to test the differences in the compression algorithms. Comparing Figure 2 to Figure 1, it is clear that all the algorithms are better at compression random sparse data, as highlighted by the results in Figure 2. It should be noted that this data isn't synthesized to play to the strengths of each of the compression methods. Adaptive Chunking still prevails as the most efficient algorithm by an extremely small margin over the vByte algorithm. While the adaptive method only prevails by a few hundredths of a percent in many cases, it is still the most efficient overall.

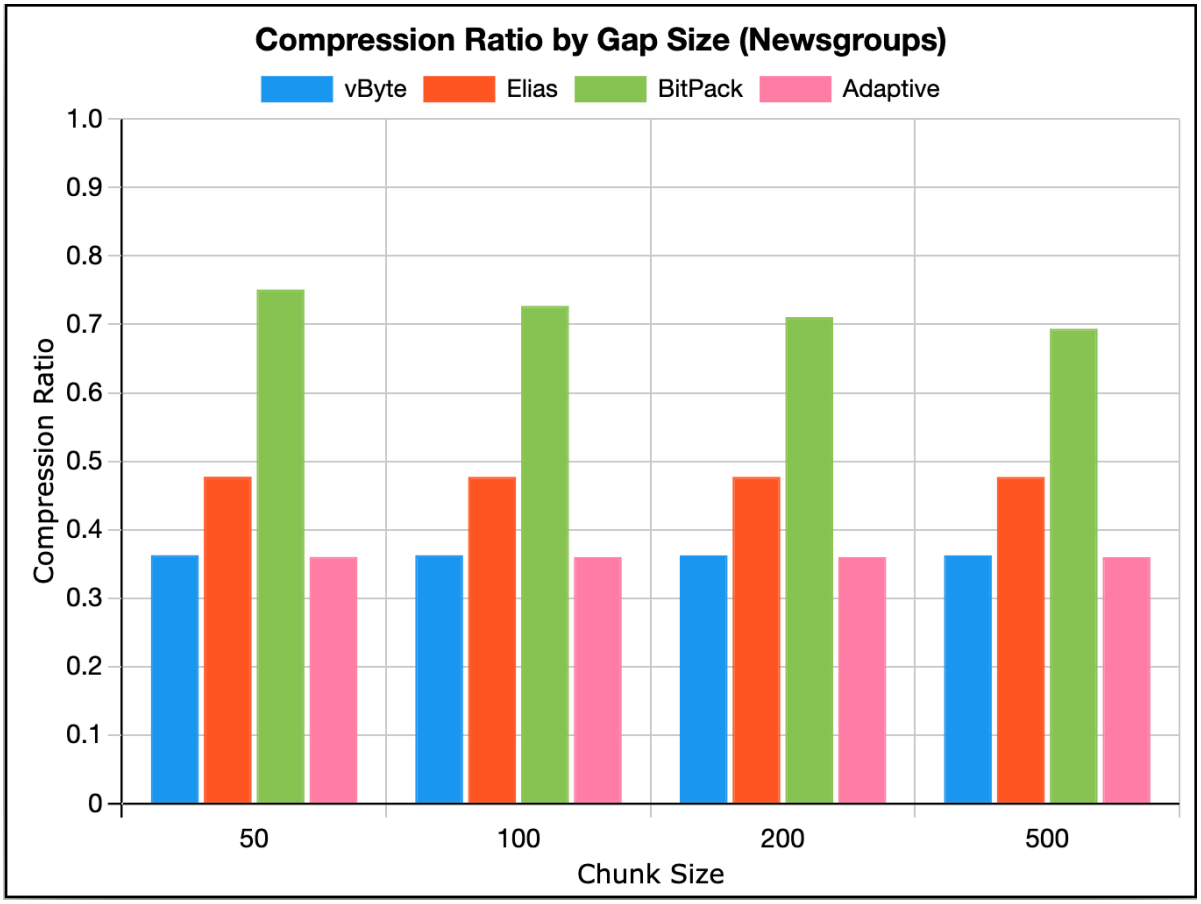


Figure 2. Compression ratio size by chuck size for each compression method

8.2 Synthetic Compression Ratio

After comparing real compression, we generated synthetic data to achieve optimal results. The results of this comparison are shown in Table 1, which gives some insight into which algorithms perform best on data of different distribution types. BitPack performs best when the data is small or clustered, significantly outperforming even Adaptive in these scenarios. But, BitPack's limitations are shown in datasets where the data is not clustered or uniform, in which it performs significant;y worse. Elia works best for datasets that are very uniform or very sparse, but struggles with clustered or monotonic distributions. vByte is consistently in the bottom two algorithms for our comparison. vByte performs well on the same data distributions as Elias, but Elias out-performs it in all of these cases. The Adaptive approach consistently performs well. While it is not always the best performing algorithm, it comes in second place in all distributions that it does not win. This makes the adaptive approach a reliable option in all scenarios.

	vByte Size	Elias	BitPack	Adaptive	Winner
Uniform Small	10,000	6,272	5,009	5,783	BitPack
Clustered	20,000	23,750	3,757	4,540	BitPack

Sparse with Outliers	10,890	6,858	21,259	7,091	Elias
Monotonic Gaps	24,541	33,543	18,759	12,034	Adaptive

Table 1. Results of comparison testing based on generated gap test data

8.3 Compression Time

Compression time results in an inherent trade-off associated with the adaptive approach. vByte was the fastest method at approximately 2.5 seconds, while Elias Gamma and BiPacking required approximately 5 and 4 seconds respectively. The adaptive algorithm incurred the highest compression time at approximately 11 seconds. The algorithm sacrifices raw compression speed in order to achieve consistently superior compression ratios across diverse data profiles.

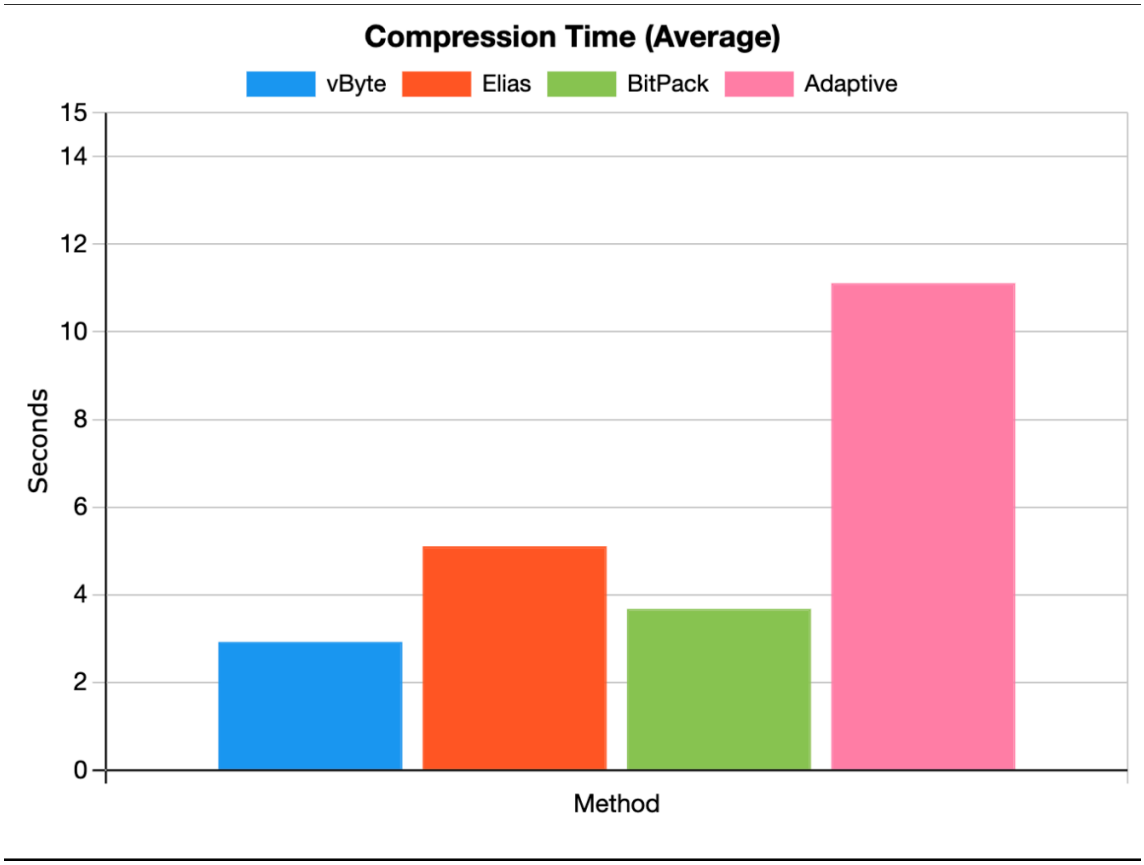


Figure 3. Results of compression time testing for each type

9. Conclusion & Future Work

This project proposed and evaluated an adaptive chuck-based compression algorithm for inverted index posting list, demonstrating that a context-aware, data-driven approach can outperform static encoding in specific workloads. By dynamically selecting among vByte, Elias gamma, and BitPacking strategies on a per-chunk basis, the algorithm avoids the one-size-fits-all limitation that characterize conventional compression methods.

Experiments across three datasets – Shakespeare, Newsgroups, and synthetically generated gap sequences – revealed that no single compression scheme dominates across all data profiles. BitPacking achieved the best compression ratio on uniform and clustered distribution, Elias Gamma excelled on sparse data with outliers and adaptive algorithm produced the smallest output on monotonic gap sequence. Critically, the Adaptive approach ranked in the top two across every distribution tested, confirming it reliability as a general-purpose solution where the optimal compression method is not known in advance,

Compression time result reveals an inherent trade-off associated with the adaptive approach. vByte was the fastest method at approximately 2.5 second, while Adaptive incurred the highest compression time, a direct consequence of evaluating multiple codecs per chunk during the profiling pass. For applications where the storage efficiency and query throughput are prioritized over compression latency, this trade-off is well justified.

Several directions remain open for extending this work. First, the chunk size is currently treated as a fixed parameter, though the evaluation demonstrates its strong influence on compression ratio. Future work should investigate dynamic chunk sizing. Second, the current adaptive algorithm selects a single compression scheme per chunk after a lightweight profiling pass, but a more principled selection mechanism could be developed. A learned model – trained on features such as mean gap value, variance, and maximum gap within a chunk– could replace the heuristic threshold logic and potentially achieve better generalization across unseen data distribution. Finally, we can scale the experiments to larger collections, which provide stronger evidence of the algorithms' practical utility and expose any scalability limitations.